REVISION CHEAT SHEET:

SQL Server Indexes

Complete notes - design - execution behavior - maintenance - practical tuning

INDEX - Main Topics & Important Subtopics

Study flow: Index purpose -> structure -> clustered/nonclustered -> composite/unique/filtered/covering -> seek/scan/lookups -> heaps -> maintenance -> practical design.

- Index Basics & Why They Matter
- Rowstore B+ Tree Structure
- Clustered Index
- Nonclustered Index
- Composite Index & Key Order
- Unique Index
- Filtered Index
- Covering Index & INCLUDE
- Seek, Scan & Residual Predicates
- Key Lookup & RID Lookup
- Heap Tables
- Fill Factor & Page Splits
- Fragmentation & Page Density
- REORGANIZE, REBUILD & Statistics
- Metadata & Index Inspection
- Practical FundraisingDb Index Design
- Index Selection Rules
- Foreign Keys & Indexes
- Common Index Mistakes
- Interview Questions
- Final Revision & Memory Rules

An **index** is a database structure that helps SQL Server find rows more efficiently.

Think about a book.

Without an index:

EXAMPLE / FLOW

```
Need information about "Transactions"
        |
        v
Read page 1
Read page 2
Read page 3
...
Read every page
```

With an index:

EXAMPLE / FLOW

```
Book Index
   |
   v
Transactions → Page 420
```

```
   |
   v
Go directly to page 420
```

SQL Server indexes work on the same general idea.

Instead of always examining every row in a table, an index can help SQL Server locate the required rows much more efficiently.

Why Do We Need Indexes?

Suppose:

EXAMPLE / FLOW
```
fundraising.Gift
```

contains:

EXAMPLE / FLOW
```
100,000,000 Gift rows
```

and we execute:

SQL
```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Without a useful index SQL Server may need to examine a very large part of the table.

Conceptually:

EXAMPLE / FLOW
```
Gift row 1
Gift row 2
Gift row 3
Gift row 4
...
Gift row 100,000,000
```

With an appropriate index on:

EXAMPLE / FLOW
```
ConstituentId
```

SQL Server may be able to navigate directly toward the relevant rows.

Indexes can therefore reduce:

EXAMPLE / FLOW
```
Logical reads

CPU
```

```
Elapsed time

Amount of data examined
```

But indexes are **not free**.

Every index also adds:

EXAMPLE / FLOW

```
Storage

INSERT overhead

UPDATE overhead

DELETE overhead

Maintenance work
```

The goal is not:

> ! Create as many indexes as possible.

The goal is:

> ! Create the smallest useful set of indexes that supports the important workload.

Main SQL Server Rowstore Index Concepts

For this chapter, focus on:

EXAMPLE / FLOW

```
Clustered Index

Nonclustered Index

Composite Index

Unique Index

Filtered Index

Covering Index

Included Columns

Heap
```

And the execution-plan behaviors:

EXAMPLE / FLOW

```
Index Seek

Index Scan

Key Lookup

RID Lookup
```

And maintenance topics:

EXAMPLE / FLOW

```
Fill Factor

Page Splits

Fragmentation

Page Density

REORGANIZE

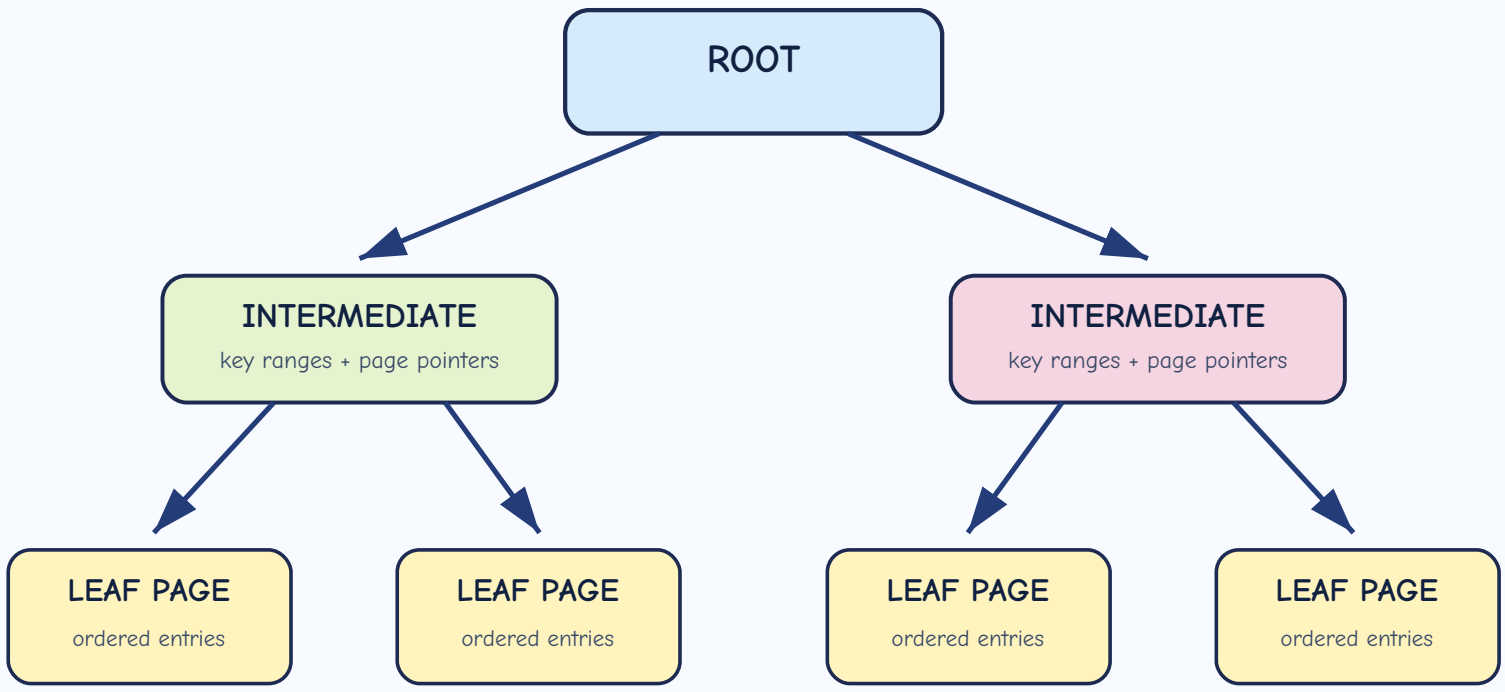
REBUILD

Statistics
```

How a SQL Server Rowstore Index Is Structured

3D DIAGRAM

Rowstore B+ Tree - Root to Leaf



SQL Server navigates root and intermediate pages to reach a much smaller leaf range instead of scanning every row.

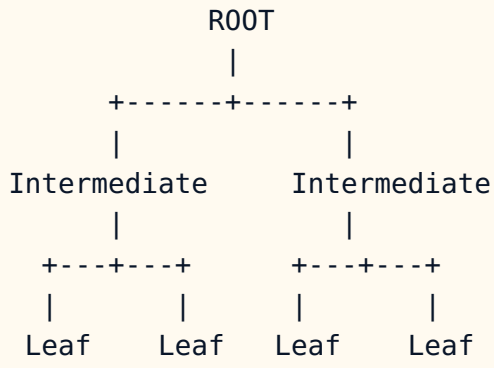
SQL Server rowstore indexes use a tree structure commonly described as a:

EXAMPLE / FLOW

```
B+ Tree
```

Conceptually:

EXAMPLE / FLOW



The tree allows SQL Server to navigate through a relatively small number of pages instead of reading every row.

Microsoft describes clustered and nonclustered rowstore indexes as B+ tree structures with root, intermediate, and leaf levels.

Clustered Index

Simple Definition

A **clustered index** determines how the table's data rows are stored according to the clustered key.

The key idea is:

EXAMPLE / FLOW

```
Clustered Index
=
The table data itself is stored at the leaf level
of the clustered index.
```

SQL Server allows only:

EXAMPLE / FLOW

```
one clustered index per table
```

because the table's rows can have only one clustered storage order.

Clustered Index Structure

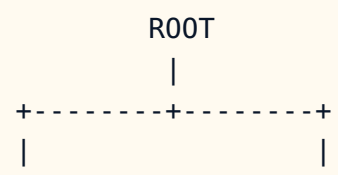
Suppose `Gift` has a clustered index on:

EXAMPLE / FLOW

```
GiftId
```

Conceptually:

EXAMPLE / FLOW



```
         GiftId 1-500      GiftId 501-1000
             |                 |
             v                 v
        Data Pages         Data Pages
             |
             v
Actual Gift rows
```

The leaf level contains:

EXAMPLE / FLOW

```
GiftId
TenantId
ConstituentId
CampaignId
GiftDate
TotalAmount
...
```

because the leaf level is the actual table data.

Clustered Index Example

Learning example:

SQL

```
CREATE UNIQUE CLUSTERED INDEX
    CX_Gift_GiftId
ON fundraising.Gift
(
    GiftId
);
```

Now:

EXAMPLE / FLOW

```
Gift
```

becomes a clustered table.

Primary Key and Clustered Index

These are related, but they are **not the same thing**.

A:

EXAMPLE / FLOW

```
PRIMARY KEY
```

is a logical integrity constraint.

A:

EXAMPLE / FLOW

```
CLUSTERED INDEX
```

is a physical access/storage structure.

When you create a **PRIMARY KEY**, SQL Server automatically creates a unique index to enforce it. If the table does not already have a clustered index and you do not explicitly request **NONCLUSTERED**, SQL Server normally uses a clustered index for the primary key.

Example

SQL

```
CREATE TABLE fundraising.ExampleGift
(
    GiftId BIGINT NOT NULL,

    CONSTRAINT PK_ExampleGift
        PRIMARY KEY CLUSTERED
        (
            GiftId
        )
);
```

But you could explicitly write:

SQL

```
PRIMARY KEY NONCLUSTERED
```

and create a different clustered index.

Therefore memorize:

EXAMPLE / FLOW

```
Primary Key
≠ Clustered Index

Primary Key
→ constraint

Clustered Index
→ storage/access structure
```

Good Clustered Index Key Properties

Microsoft's current design guidance favors clustered keys that are:

EXAMPLE / FLOW

```
Narrow

Unique

Ever-increasing when practical

Stable / rarely changed
```

A narrow clustered key matters because the clustered key is also stored as the row locator in nonclustered indexes on that table.

Narrow Clustered Key

Suppose clustered key is:

EXAMPLE / FLOW

```
GiftId BIGINT
```

That is relatively narrow.

Now imagine instead using:

EXAMPLE / FLOW

```
TenantId
+
CampaignCode
+
ConstituentEmail
+
GiftDate
```

as a huge clustered key.

Every nonclustered index may need to carry that clustering key as the row locator.

Result:

EXAMPLE / FLOW

```
Wider nonclustered indexes

More storage

More memory

More I/O
```

Therefore:

! Keep clustered keys reasonably narrow.

Unique Clustered Key

Suppose:

EXAMPLE / FLOW

```
GiftId
```

is unique.

Excellent.

If a clustered index itself is not unique, SQL Server may internally add a hidden uniqueifier to distinguish duplicate clustered-key rows.

Therefore a naturally unique clustering key is usually desirable.

Stable Clustered Key

Suppose the clustered key changes frequently.

Because nonclustered indexes use the clustered key as the row locator, changing the clustered key can require additional index maintenance.

That is why an immutable or rarely changed clustered key is usually preferable.

Increasing Clustered Key

A sequential key such as:

EXAMPLE / FLOW

```
GiftId = 10000
GiftId = 10001
GiftId = 10002
GiftId = 10003
```

usually inserts new rows toward the end of the index.

This can reduce middle-of-index page splits compared with highly random insertion patterns.

Do not memorize:

EXAMPLE / FLOW

```
Sequential key is always perfect
```

because extremely high concurrent insert workloads can have separate last-page contention considerations.

But for a beginner:

> ! A narrow, unique, stable, sequential key is often a strong clustered-key design.

Does a Clustered Index Guarantee SELECT Order?

No.

This is very important.

Even though the clustered index stores rows according to the clustering key, SQL Server does not guarantee result-set ordering unless you write:

SQL

```
ORDER BY
```

Always use:

SQL

```
ORDER BY GiftId;
```

when output order matters.

Only One Clustered Index

Suppose you want:

EXAMPLE / FLOW

```
GiftId order

GiftDate order

ConstituentId order
```

A table cannot have three clustered indexes.

You choose one clustering key.

Other access paths are usually supported with:

EXAMPLE / FLOW

```
nonclustered indexes.
```

When Is a Clustered Index Useful?

Often useful when:

EXAMPLE / FLOW

```
The table is frequently queried

Range queries are important

Rows need a stable clustering key

The key is narrow and stable

The table should not remain a heap
```

Microsoft notes that, with exceptions, most tables should generally have a carefully selected clustered index rather than remain heaps.

Nonclustered Index

Simple Definition

A **nonclustered index** is a separate structure from the table data.

Think:

EXAMPLE / FLOW

```
Table data
      |
      |
Nonclustered Index
contains:
key values
+
row locator
```

Microsoft describes nonclustered indexes as separate structures containing index keys plus row locators pointing back to the base row.

Example

Suppose we frequently execute:

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Learning index:

```sql
CREATE INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
);
```

Now SQL Server has a searchable structure ordered by:

EXAMPLE / FLOW

```
ConstituentId
```

Nonclustered Index Concept

EXAMPLE / FLOW

```
Nonclustered Index

ConstituentId
-------------
1000
1000
1001  ──────────────┐
1001                │
1002                │
                    │
                    v
               Actual Gift rows
```

Row Locator

Every nonclustered index needs a way to find the real row.

The locator depends on the base table.

If the table has a clustered index:

EXAMPLE / FLOW

```
Nonclustered index row locator
=
Clustered index key
```

If the table is a heap:

EXAMPLE / FLOW

```
Nonclustered index row locator
=
RID
```

Microsoft documents this distinction explicitly.

Example with Clustered Gift Table

Suppose:

EXAMPLE / FLOW

```
Clustered key = GiftId
```

and nonclustered index:

EXAMPLE / FLOW

```
IX_Gift_ConstituentId
```

Conceptually the leaf-level entries contain:

EXAMPLE / FLOW

```
ConstituentId
+
GiftId row locator
```

Example:

EXAMPLE / FLOW

```
ConstituentId   GiftId
-------------   ------
1001            10000
1001            10125
1001            10990
```

SQL Server can then use `GiftId` to locate the complete clustered row if more columns are required.

How Many Nonclustered Indexes?

Unlike clustered indexes, a table can have many nonclustered indexes.

Current SQL Server supports up to 999 nonclustered indexes per table, although creating anywhere near that number would normally be an extremely poor application design.

Practical rule:

> ! Create only indexes justified by real workload requirements.

Cost of Nonclustered Indexes

Suppose `Gift` has:

EXAMPLE / FLOW

```
10 nonclustered indexes
```

Every:

SQL

```
INSERT INTO fundraising.Gift ...
```

may need to update:

EXAMPLE / FLOW

```
base clustered table
+
10 index structures
```

Likewise updates and deletes may require index maintenance.

Therefore:

EXAMPLE / FLOW

```
More indexes
→ potentially faster reads

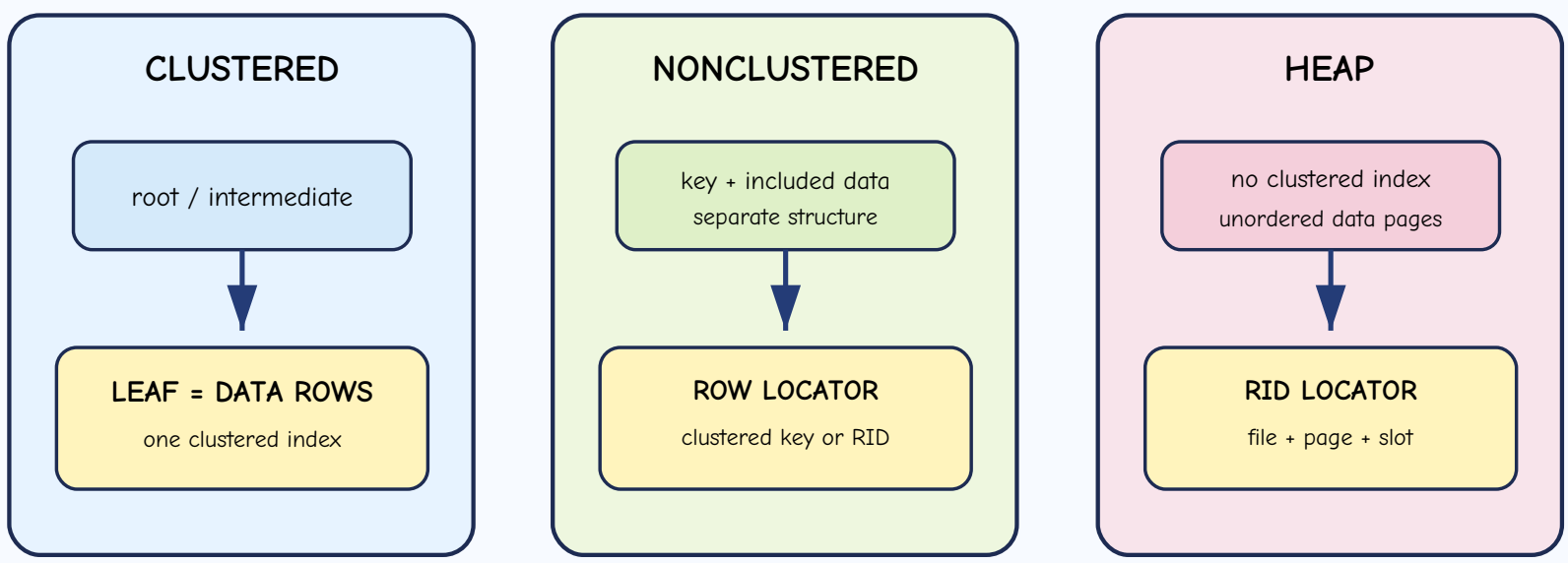
but

More indexes
→ more write/storage/maintenance cost
```

Clustered vs Nonclustered Index

3D DIAGRAM

Clustered vs Nonclustered vs Heap



Clustered tables store data at the leaf; nonclustered indexes point back to rows; heaps use RIDs because no clustered key exists.

Clustered Index	Nonclustered Index
Leaf level contains data rows	Separate structure
One per table	Many possible
Defines clustering of table rows	Provides additional access paths
Row itself is at leaf	Leaf contains key + locator + includes
Clustered key becomes locator for NCIs	Uses clustered key or RID to find row

Easy memory:

EXAMPLE / FLOW

```
Clustered
→ data is the leaf

Nonclustered
→ separate lookup structure
```

Composite Index

Simple Definition

A **composite index** is an index whose key contains more than one column.

Example:

SQL

```
CREATE INDEX
    IX_Gift_Tenant_Constituent_Date
ON fundraising.Gift
(
    TenantId,
    ConstituentId,
    GiftDate
);
```

This index has three key columns.

Why Use a Composite Index?

Suppose a normal multi-tenant API query is:

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TenantId = 1
  AND ConstituentId = 1001
  AND GiftDate >= '2026-01-01';
```

A composite key:

EXAMPLE / FLOW

```
TenantId
→ ConstituentId
→ GiftDate
```

can align well with the query.

Composite Index Key Order Matters

This is one of the most important index rules.

Index:

EXAMPLE / FLOW

```
(TenantId, ConstituentId, GiftDate)
```

is **not equivalent** to:

EXAMPLE / FLOW

```
(GiftDate, ConstituentId, TenantId)
```

The index tree is ordered starting from the first key.

Think of a phone directory:

EXAMPLE / FLOW

```
LastName
then
FirstName
```

It is easy to search:

EXAMPLE / FLOW

```
LastName = 'Smith'
```

or:

EXAMPLE / FLOW

```
LastName = 'Smith'
AND FirstName = 'John'
```

But searching only:

EXAMPLE / FLOW

```
FirstName = 'John'
```

is much less naturally supported.

Microsoft emphasizes that composite key order affects whether an index can efficiently support search predicates.

Leading-Key Concept

For:

EXAMPLE / FLOW

```
(TenantId, ConstituentId, GiftDate)
```

queries beginning with:

EXAMPLE / FLOW

```
TenantId
```

are naturally aligned with the index.

For example:

SQL

```
WHERE TenantId = 1
```

or:

SQL

```
WHERE TenantId = 1
  AND ConstituentId = 1001
```

or:

SQL

```
WHERE TenantId = 1
  AND ConstituentId = 1001
  AND GiftDate >= '2026-01-01'
```

But:

SQL

```
WHERE ConstituentId = 1001
```

without `TenantId` may not support an efficient seek using the same composite index.

SQL Server might still scan the index, but the useful seek capability is reduced.

Equality Before Range - Practical Design Rule

Consider:

SQL

```
WHERE TenantId = @TenantId
  AND ConstituentId = @ConstituentId
  AND GiftDate >= @StartDate
```

A useful candidate order is often:

EXAMPLE / FLOW

```
TenantId
ConstituentId
GiftDate
```

because:

EXAMPLE / FLOW

```
TenantId
→ equality

ConstituentId
→ equality

GiftDate
→ range
```

Once SQL Server reaches a broad range portion of the key, later columns may become less useful for narrowing the seek.

This is a practical design guideline, not a blind universal formula.

Composite Index and ORDER BY

Suppose:

SQL

```sql
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TenantId = 1
  AND ConstituentId = 1001
ORDER BY GiftDate DESC;
```

An index such as:

SQL

```sql
CREATE INDEX IX_Gift_Tenant_Constituent_GiftDate
ON fundraising.Gift
(
    TenantId,
    ConstituentId,
    GiftDate DESC
);
```

may help SQL Server both:

EXAMPLE / FLOW

```
find the required rows
```

and potentially:

EXAMPLE / FLOW

```
avoid a separate Sort operator
```

depending on the complete query.

Unique Index

Simple Definition

A **unique index** guarantees that no duplicate index-key values exist.

Example:

```sql
CREATE UNIQUE INDEX
    UX_Campaign_Tenant_CampaignCode
ON fundraising.Campaign
(
    TenantId,
    CampaignCode
);
```

Now within this design:

```
Tenant 1 + CampaignCode 'CANCER2026'
```

can appear only once.

Why Use a Unique Index?

It serves two purposes:

```
1. Enforces data integrity

2. Gives the optimizer useful knowledge
   that values are unique
```

Microsoft notes that unique indexes guarantee uniqueness and provide additional information that can help the optimizer.

Unique Index vs UNIQUE Constraint

Both can enforce uniqueness.

Example constraint:

```sql
ALTER TABLE fundraising.Campaign
ADD CONSTRAINT UQ_Campaign_Tenant_Code
UNIQUE
(
    TenantId,
    CampaignCode
);
```

SQL Server uses a unique index to enforce the constraint.

When the requirement is clearly:

> ! This is a business uniqueness rule.

A:

```
UNIQUE constraint
```

often communicates the intent more clearly.

When the index exists mainly as an indexing design choice, a standalone:

EXAMPLE / FLOW
```
UNIQUE INDEX
```

may be appropriate.

The optimizer does not fundamentally treat a manually created unique index differently merely because one came from a UNIQUE constraint.

Unique Index and NULL

SQL Server treats `NULL` values as equal for unique-index enforcement.

Therefore, a single-column unique index generally permits only:

EXAMPLE / FLOW
```
one NULL
```

in that indexed key.

This surprises many developers.

Filtered Unique Index for Optional Email

Suppose:

EXAMPLE / FLOW
```
Constituent.PrimaryEmail
```

can be `NULL`.

You may want:

> ! Emails must be unique within a tenant when an email actually exists.

A learning-only candidate:

SQL
```sql
CREATE UNIQUE INDEX
    UX_Constituent_Tenant_PrimaryEmail
ON constituent.Constituent
(
    TenantId,
    PrimaryEmail
)
WHERE PrimaryEmail IS NOT NULL;
```

This allows:

EXAMPLE / FLOW
```
many NULL email values
```

because those rows are excluded from the index.

But populated email values must remain unique per tenant.

This is a powerful use of:

EXAMPLE / FLOW

```
UNIQUE
+
FILTERED INDEX
```

Filtered Index

Simple Definition

A **filtered index** is a nonclustered index containing only rows that satisfy a filter condition.

Example:

SQL

```sql
CREATE INDEX
    IX_Task_OpenTasks
ON engagement.Task
(
    TenantId,
    AssignedToUserId,
    DueDate
)
WHERE CompletedAtUtc IS NULL;
```

This index contains only:

EXAMPLE / FLOW

```
open tasks
```

rather than every Task row.

Filtered indexes are optimized nonclustered indexes designed for well-defined subsets of rows.

Why Filter an Index?

Imagine:

EXAMPLE / FLOW

```
Task table
= 100 million rows

Completed tasks
= 98 million

Open tasks
= 2 million
```

If most application queries are:

SQL

```sql
WHERE CompletedAtUtc IS NULL
```

why index all 100 million rows?

Filtered index:

EXAMPLE / FLOW

```
Only open Task rows
```

can provide:

EXAMPLE / FLOW

```
Smaller index

Less storage

Lower update maintenance for excluded rows

More focused statistics

Potentially faster matching queries
```

Filtered Index Example - Unprocessed Outbox Messages

Your future:

EXAMPLE / FLOW

```
integration.OutboxMessage
```

table could naturally have:

EXAMPLE / FLOW

```
ProcessedAtUtc
```

where unprocessed rows are:

EXAMPLE / FLOW

```
ProcessedAtUtc IS NULL
```

Learning index:

SQL

```
CREATE INDEX
    IX_Outbox_Unprocessed
ON integration.OutboxMessage
(
    TenantId,
    OccurredAtUtc
)
WHERE ProcessedAtUtc IS NULL;
```

This is a classic filtered-index pattern.

Filtered Index Rules

Filtered indexes:

EXAMPLE / FLOW

```
Are nonclustered indexes

Use a WHERE-style filter predicate

Work best for well-defined subsets
```

```
Support simple comparison predicates

Do not support arbitrary multi-table conditions
```

Microsoft notes that filtered-index predicates are defined on one table and support relatively simple comparison logic; `LIKE` is not supported in the filter expression.

When to Use Filtered Index

Good examples:

EXAMPLE / FLOW

```
CompletedAtUtc IS NULL

Status = 'Active'

ProcessedAtUtc IS NULL

IsDeleted = 0

Optional column IS NOT NULL
```

provided those subsets are important to real queries.

When Not to Use Filtered Index

Avoid when:

EXAMPLE / FLOW

```
Almost every row qualifies

Queries don't use a compatible predicate

The filter condition changes constantly

The filter requires complex multi-table logic
```

Covering Index

Simple Definition

A **covering index is not a separate physical index type**.

It means:

> ! The index contains everything a particular query needs.

The required columns may be present as:

EXAMPLE / FLOW

```
Index key columns
```

```
or

Included columns
```

If the index contains all needed values, SQL Server may not have to access the base clustered index or heap.

Microsoft describes an index containing all columns referenced by a query as covering that query.

Covering Index Example

Query:

```sql
SELECT
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

Basic index:

```sql
CREATE INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
);
```

This can help find the rows.

But the query also needs:

```
GiftDate

TotalAmount
```

A covering design could be:

```sql
CREATE INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
)
INCLUDE
(
    GiftDate,
    TotalAmount
);
```

Now the index contains:

```
Search key:
ConstituentId
```

```
Output data:
GiftDate
TotalAmount
```

INCLUDE Columns

Simple Definition

`INCLUDE` adds nonkey columns to the leaf level of a nonclustered index.

Example:

SQL

```
CREATE INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
)
INCLUDE
(
    GiftDate,
    TotalAmount,
    CampaignId
);
```

Microsoft recommends included columns when values are needed by a query for output but do not need to participate in searching, grouping, or ordering.

Key Columns vs Included Columns

This distinction is extremely important.

Consider:

SQL

```
CREATE INDEX IX_Gift_Search
ON fundraising.Gift
(
    TenantId,
    ConstituentId
)
INCLUDE
(
    GiftDate,
    TotalAmount
);
```

Key:

EXAMPLE / FLOW

```
TenantId
ConstituentId
```

Used for:

```
EXAMPLE / FLOW
Searching

Joining

Ordering

Range navigation
```

Included:

```
EXAMPLE / FLOW
GiftDate
TotalAmount
```

Stored mainly so SQL Server can return them without another lookup.

Visual Structure

```
EXAMPLE / FLOW
Nonclustered Index

Key:
TenantId
ConstituentId

Leaf also contains:
GiftDate
TotalAmount
Clustered row locator
```

Included Columns Are Not Part of Search Order

Suppose:

```
EXAMPLE / FLOW
TotalAmount
```

is only an included column.

Query:

```
SQL
WHERE TotalAmount > 10000
```

cannot usually seek through the index based on `TotalAmount` because it is not part of the index key ordering.

Included columns:

```
EXAMPLE / FLOW
help cover
```

but do not provide the same searching capability as key columns.

INCLUDE Column Order

The order of columns inside:

```
SQL
INCLUDE (...)
```

does not affect query searching performance because included columns are nonkey leaf-level data.

Why Not Make Every Column a Key?

Because key columns:

```
EXAMPLE / FLOW
Increase tree width

Increase key size

Participate in ordering

Increase maintenance/storage cost
```

If a column is only needed for:

```
EXAMPLE / FLOW
SELECT output
```

`INCLUDE` is often better.

Do Not Create Extremely Wide Covering Indexes

Example:

```
EXAMPLE / FLOW
INCLUDE 25 very wide columns
```

may eliminate lookups but create a huge index.

Wide covering indexes can:

```
EXAMPLE / FLOW
Consume more storage

Reduce rows per page

Increase I/O

Increase memory usage

Increase INSERT/UPDATE/DELETE cost
```

Microsoft explicitly recommends avoiding unnecessarily wide covering indexes.

Composite vs Covering Index

Composite:

```
EXAMPLE / FLOW
Multiple KEY columns
```

Example:

```
EXAMPLE / FLOW
(TenantId, ConstituentId, GiftDate)
```

Covering:

```
EXAMPLE / FLOW
Contains all columns needed
for one query
```

Example:

```
EXAMPLE / FLOW
KEY:
(TenantId, ConstituentId)

INCLUDE:
(GiftDate, TotalAmount)
```

An index can be:

```
EXAMPLE / FLOW
composite
AND
covering
```

at the same time.

Index Seek — Simple Definition

An **Index Seek** means SQL Server uses the ordered index structure to navigate to the required key/range.

Example query:

```
SQL
SELECT
    GiftId,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

With an appropriate index:

```
EXAMPLE / FLOW
Root
 |
 v
Intermediate
 |
 v
```

```
ConstituentId 1001 range
 |
 v
Matching rows
```

Microsoft defines Index Seek as using seek predicates to retrieve only rows matching the searchable index ranges.

Is an Index Seek Always Good?

No.

Suppose:

EXAMPLE / FLOW
```
ConstituentId = 1001
```

returns:

EXAMPLE / FLOW
```
5 rows
```

A seek is likely excellent.

But if it returns:

EXAMPLE / FLOW
```
50 million rows
```

the seek still has to process 50 million rows.

Therefore:

EXAMPLE / FLOW
```
Seek
≠ automatically fast
```

Always ask:

EXAMPLE / FLOW
```
How many rows?

How many logical reads?

Are lookups happening?

How selective is the predicate?
```

Seek Predicate vs Residual Predicate

A seek can sometimes have both:

EXAMPLE / FLOW
```
Seek Predicate
```

and:

EXAMPLE / FLOW

```
Predicate
```

The seek predicate determines what SQL Server can navigate through the index to locate.

A residual predicate may then filter rows afterward.

Conceptually:

EXAMPLE / FLOW

```
Seek 10,000 rows
      |
      v
Predicate filters
      |
      v
Return 10 rows
```

If many rows are read but few returned, index design or predicate design may deserve investigation.

Index Scan

Simple Definition

An **Index Scan** reads a broad portion or all of an index.

Example:

SQL

```
SELECT
    SUM(TotalAmount)
FROM fundraising.Gift;
```

There is no selective filter.

SQL Server may need to read all Gift values.

A scan may therefore be exactly correct.

Microsoft describes Index Scan as reading rows from an index and optionally evaluating a predicate.

Seek vs Scan

EXAMPLE / FLOW

```
Index Seek
→ Navigate to specific key/range

Index Scan
→ Read broad/all index structure
```

Is Index Scan Bad?

No.

Good reasons for a scan include:

EXAMPLE / FLOW

```
Query needs most rows

Table/index is small

Aggregation needs the entire set

Scan of a narrow index is cheaper
than another access path
```

The beginner rule:

> ! Don't tune based only on whether the plan says Seek or Scan.

Clustered Index Scan

A:

EXAMPLE / FLOW

```
Clustered Index Scan
```

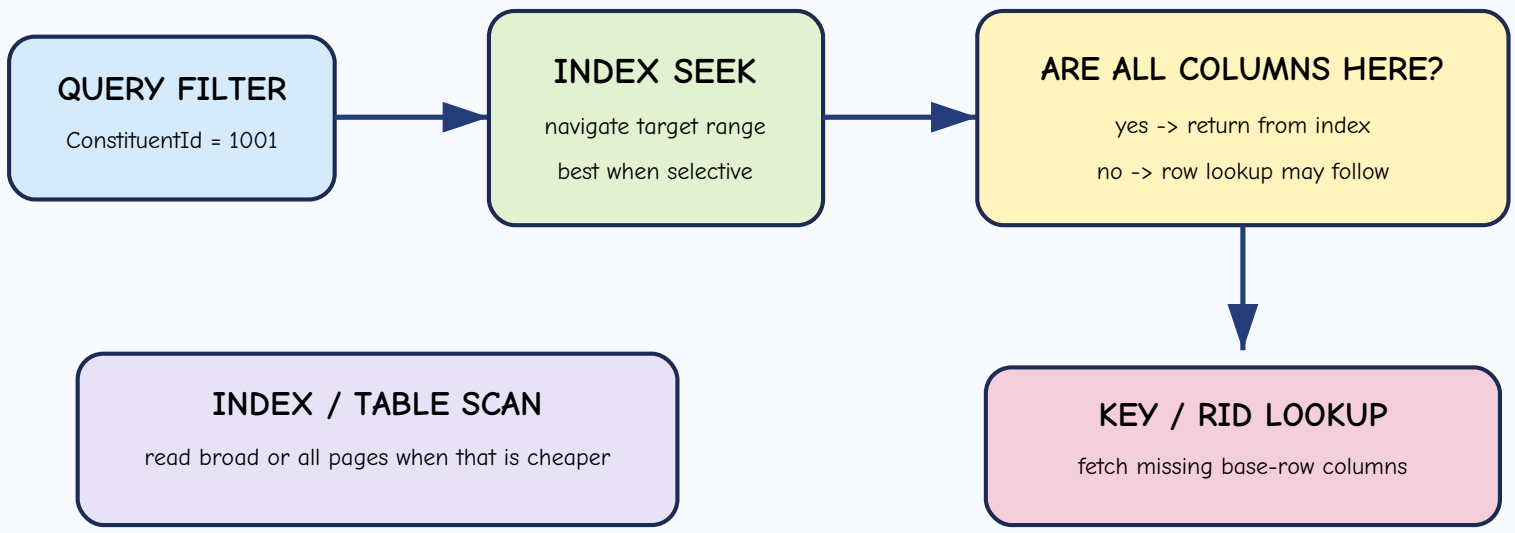
means SQL Server is scanning the clustered table structure.

Because the clustered leaf level contains the data rows, this can effectively mean reading much or all of the table.

Key Lookup

FLOW DIAGRAM

Seek, Scan and Lookup - What SQL Server Does



A seek is not automatically good and a scan is not automatically bad; row counts, logical reads, and lookup repetitions determine the real cost.

Simple Definition

A **Key Lookup** occurs when SQL Server:

```
EXAMPLE / FLOW
1. Uses a nonclustered index to find rows

2. Needs additional columns not stored there

3. Looks up each corresponding clustered row
```

Microsoft defines Key Lookup as a bookmark lookup into a clustered index using the clustered row locator.

Key Lookup Example

Suppose index:

```
SQL
CREATE INDEX
    IX_Gift_Constituent
ON fundraising.Gift
(
    ConstituentId
);
```

Query:

```
SQL
SELECT
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE ConstituentId = 1001;
```

The nonclustered index knows:

```
EXAMPLE / FLOW
ConstituentId
+
clustered row locator
```

But it may not contain:

```
EXAMPLE / FLOW
GiftDate
TotalAmount
```

Plan:

```
EXAMPLE / FLOW
Index Seek
    |
    v
Matching index rows
    |
    v
Nested Loops
    |
    v
Key Lookup
    |
```

```
    v
Fetch GiftDate + TotalAmount
```

Key Lookup with Few Rows

Suppose:

EXAMPLE / FLOW
```
3 matching Gift rows
```

Then:

EXAMPLE / FLOW
```
3 Key Lookups
```

may be extremely efficient.

Key Lookup with Many Rows

Suppose:

EXAMPLE / FLOW
```
500,000 matching Gift rows
```

Then SQL Server may perform:

EXAMPLE / FLOW
```
500,000 lookups
```

Conceptually:

EXAMPLE / FLOW
```
Seek
 |
 v
500,000 row locators
 |
 v
Lookup
Lookup
Lookup
Lookup
...
500,000 times
```

That can generate high I/O.

Removing a Key Lookup with a Covering Index

Original:

```sql
CREATE INDEX IX_Gift_Constituent
ON fundraising.Gift
(
    ConstituentId
);
```

Potential covering version:

```sql
CREATE INDEX IX_Gift_Constituent
ON fundraising.Gift
(
    ConstituentId
)
INCLUDE
(
    GiftDate,
    TotalAmount
);
```

Then SQL Server may get everything directly from the nonclustered index.

Plan might become:

```
Index Seek
     |
     v
Result
```

instead of:

```
Index Seek
     |
     v
Key Lookup
```

Should Every Key Lookup Be Removed?

No.

A lookup repeated:

```
2 times
```

is probably harmless.

Do not create a large index just to remove a tiny lookup.

Always compare:

```
Lookup cost

vs
```

```
Additional index storage/write cost
```

Heap — Simple Definition

A **heap** is a table without a clustered index.

Example:

EXAMPLE / FLOW

```
Table exists

but

no clustered index exists
```

Therefore the data rows are not organized by a clustered key.

Microsoft defines a heap as a table without a clustered index.

Heap Structure

Conceptually:

EXAMPLE / FLOW

```
Heap
 |
 +---- Data Page
 |
 +---- Data Page
 |
 +---- Data Page
 |
 +---- Data Page
```

Rows are not maintained in clustered-key order.

RID - Row Identifier

Rows in a heap are addressed using a:

EXAMPLE / FLOW

```
RID
```

which identifies:

EXAMPLE / FLOW

```
File

Page

Slot
```

Microsoft describes the heap RID as an 8-byte identifier consisting of file, page, and slot location.

Nonclustered Index on Heap

Suppose a heap has:

```sql
CREATE INDEX IX_HeapGift_ConstituentId
ON fundraising.SomeHeapGift
(
    ConstituentId
);
```

The nonclustered row locator is:

```
RID
```

rather than a clustered key.

RID Lookup

Simple Definition

A **RID Lookup** is similar to a Key Lookup, but it retrieves the base row from a heap using the RID.

Microsoft describes RID Lookup as a bookmark lookup into a heap using the row identifier.

Flow:

```
Nonclustered Index Seek
        |
        v
Find RID
        |
        v
RID Lookup
        |
        v
Heap row
```

Key Lookup vs RID Lookup

```
Clustered table
→ Key Lookup

Heap
→ RID Lookup
```

Memory rule:

```
KEY Lookup
→ clustered key locator

RID Lookup
→ heap row identifier
```

Heap Table Scan

If a heap has no useful nonclustered index and SQL Server needs rows:

EXAMPLE / FLOW

```
Table Scan
```

may occur.

Microsoft notes that a large heap without useful nonclustered indexes may need a complete table scan to find rows.

When Can a Heap Be Useful?

Common specialized cases:

EXAMPLE / FLOW

```
Staging tables

Temporary load tables

Tables frequently truncated and reloaded

Large unordered bulk-load workflows
```

Microsoft specifically lists staging/reload scenarios as reasonable heap use cases.

When Should a Heap Usually Be Avoided?

A heap is often a poor fit when:

EXAMPLE / FLOW

```
Rows are frequently updated

Range queries are common

Ordering/grouping is important

The table is large and frequently queried

There are no useful nonclustered indexes
```

Forwarded Records in Heaps

Suppose a heap row becomes larger after an update.

If it no longer fits on its page, SQL Server may move it elsewhere and leave a forwarding pointer.

Conceptually:

EXAMPLE / FLOW

```
Original Page
    |
    | forwarding pointer
```

```
    v
New Page
    |
    v
Actual row
```

Many forwarded records can increase I/O during heap scans. Microsoft warns that frequent heap updates can create forwarded records that reduce read-ahead efficiency and add extra I/O.

Unique + Composite + Filtered + Covering

These terms can combine.

Example:

SQL

```
CREATE UNIQUE INDEX
    UX_Constituent_Email
ON constituent.Constituent
(
    TenantId,
    PrimaryEmail
)
INCLUDE
(
    DisplayName
)
WHERE PrimaryEmail IS NOT NULL;
```

This single index is:

EXAMPLE / FLOW

```
Nonclustered
+
Composite
+
Unique
+
Filtered
+
Potentially covering
```

depending on the query.

This is important:

> ! Index categories are not always mutually exclusive.

Fill Factor

Simple Definition

Fill factor controls approximately how full SQL Server makes leaf-level index pages when an index is created or rebuilt.

Example:

EXAMPLE / FLOW
```
FILLFACTOR = 100
```

means SQL Server tries to fill leaf pages very tightly during the build.

Example:

EXAMPLE / FLOW
```
FILLFACTOR = 80
```

means approximately:

EXAMPLE / FLOW
```
80% used

20% reserved free space
```

at rebuild/create time.

Microsoft states that fill factor reserves free space on index pages to accommodate future growth.

Important Fill Factor Rule

Fill factor is **not continuously maintained**.

Suppose an index is rebuilt with:

EXAMPLE / FLOW
```
FILLFACTOR = 80
```

Pages initially have space.

As inserts and updates happen:

EXAMPLE / FLOW
```
free space gets consumed
```

SQL Server does not constantly keep every page at exactly 80% fullness.

Default Fill Factor

SQL Server's server default is commonly:

EXAMPLE / FLOW
```
0
```

which for leaf-level filling behaves like filling pages to capacity, similar to `100`.

Why Lower Fill Factor?

Consider an index on a random key.

Pages:

EXAMPLE / FLOW

```
Page 1 = full
Page 2 = full
Page 3 = full
```

A new key must be inserted into the middle of:

EXAMPLE / FLOW

```
Page 2
```

but Page 2 is full.

SQL Server may have to split the page.

A lower fill factor can leave space for future middle-of-page inserts.

Page Split

Simple Definition

A **page split** happens when SQL Server needs space on a full index page and redistributes rows between pages.

Conceptually:

EXAMPLE / FLOW

```
Before

Page A
[100% full]

New row must go here
      |
      v

Page split

Page A       Page B
[rows]       [rows]
```

Page splits consume resources and can contribute to lower page density and fragmentation.

Should We Set FILLFACTOR = 70 Everywhere?

No.

This is a common mistake.

Suppose 30% is left empty on every page.

Then SQL Server may need:

EXAMPLE / FLOW

```
more pages

more memory

more I/O

more storage
```

to hold the same rows.

Microsoft currently recommends leaving fill factor at the default for most workloads and changing it only when measured page-split behavior justifies it.

Sequential GiftId Example

Suppose:

EXAMPLE / FLOW
```
GiftId
```

is sequential:

EXAMPLE / FLOW
```
10000
10001
10002
10003
```

New values mostly go to:

EXAMPLE / FLOW
```
the end of the index
```

Leaving 20% empty on every old page may provide little benefit because future rows are not inserted throughout those pages.

Therefore lower fill factor is usually more relevant to indexes where inserts occur:

EXAMPLE / FLOW
```
throughout the key range
```

rather than only at the end.

Index Fragmentation

Simple Definition

Index fragmentation generally describes how index pages have become less efficiently organized over time.

Two related concepts matter:

EXAMPLE / FLOW
```
Logical fragmentation

Page density
```

Logical Fragmentation

Conceptually desired page order:

EXAMPLE / FLOW
```
Page 1
  ↓
Page 2
  ↓
Page 3
```

```
  ↓
Page 4
```

Fragmented physical organization might resemble:

EXAMPLE / FLOW

```
Page 1
  ↓
Page 9
  ↓
Page 3
  ↓
Page 15
```

This can reduce the effectiveness of large ordered scans/read-ahead.

Page Density

Page density describes:

> ! How full the pages are.

Example:

EXAMPLE / FLOW

```
Page A = 95% full

Page B = 94% full

Page C = 96% full
```

High density means fewer pages are required.

Low density:

EXAMPLE / FLOW

```
Page A = 45%

Page B = 42%

Page C = 48%
```

means more pages may be required for the same rows.

Why Page Density Matters

Suppose:

EXAMPLE / FLOW

```
1,000,000 rows
```

fit into:

EXAMPLE / FLOW

```
10,000 pages
```

at high density.

If page density falls greatly, the same rows might need:

EXAMPLE / FLOW

```
18,000 pages.
```

Queries then potentially perform much more I/O.

Microsoft's current guidance emphasizes that in many workloads page density can have a greater performance impact than fragmentation percentage alone.

Measure Fragmentation and Page Density

SQL Server provides:

EXAMPLE / FLOW

```
sys.dm_db_index_physical_stats
```

Example:

SQL

```
SELECT
    OBJECT_SCHEMA_NAME(ips.object_id)
        AS SchemaName,

    OBJECT_NAME(ips.object_id)
        AS TableName,

    i.name
        AS IndexName,

    ips.page_count,

    ips.avg_fragmentation_in_percent,

    ips.avg_page_space_used_in_percent

FROM sys.dm_db_index_physical_stats
(
    DB_ID(),
    NULL,
    NULL,
    NULL,
    'LIMITED'
) AS ips

INNER JOIN sys.indexes AS i
    ON i.object_id = ips.object_id
   AND i.index_id  = ips.index_id

WHERE ips.index_id > 0

ORDER BY
    ips.avg_fragmentation_in_percent DESC;
```

Important values:

EXAMPLE / FLOW

```
avg_fragmentation_in_percent

avg_page_space_used_in_percent

page_count
```

Microsoft specifically recommends considering both fragmentation and page density when planning index maintenance.

Small Indexes and Fragmentation

Suppose an index contains:

EXAMPLE / FLOW

```
8 pages
```

and reports:

EXAMPLE / FLOW

```
70% fragmentation.
```

That number may not matter much because the structure is tiny.

Do not perform expensive maintenance simply because the percentage looks high.

Also consider:

EXAMPLE / FLOW

```
page_count

query usage

performance impact
```

Traditional 5% / 30% Rule

You may encounter older scripts using rules such as:

EXAMPLE / FLOW

```
< 5%
→ nothing

5% to 30%
→ reorganize

> 30%
→ rebuild
```

Do not memorize this as a SQL Server law.

Modern Microsoft guidance recommends workload-driven maintenance rather than automatically reorganizing/rebuilding every index using fixed fragmentation thresholds.

Use thresholds only as:

EXAMPLE / FLOW

```
an operational starting policy
```

that has been validated for your environment.

Index Maintenance

The two main rowstore maintenance operations are:

EXAMPLE / FLOW

```
REORGANIZE

REBUILD
```

REORGANIZE

Simple Definition

REORGANIZE incrementally defragments the leaf level of an index.

Example:

SQL

```
ALTER INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
REORGANIZE;
```

Microsoft describes reorganizing as less resource intensive than rebuilding and always an online operation.

REORGANIZE Characteristics

EXAMPLE / FLOW

```
Less intensive

Operates incrementally

Defragments leaf pages

Online operation

Does not recreate entire index
```

It can also compact pages.

REBUILD

Simple Definition

REBUILD recreates the index structure.

Example:

SQL

```
ALTER INDEX
    IX_Gift_ConstituentId
```

```
ON fundraising.Gift
REBUILD;
```

Conceptually:

EXAMPLE / FLOW

```
Old index
   |
   v
Build new index structure
   |
   v
Replace old structure
```

REBUILD Characteristics

Generally:

EXAMPLE / FLOW

```
More resource intensive

Can need considerable CPU/I/O

Can require additional space

Can reset fill factor

Can be online in supported scenarios/options

Refreshes index statistics as part of rebuild
```

Microsoft documents `ALTER INDEX ... REBUILD` as re-creating the index and notes that rebuild refreshes the related index statistics; `REORGANIZE` does not update those statistics.

REORGANIZE vs REBUILD

REORGANIZE	REBUILD
Incremental	Recreates index
Lower resource cost	Higher resource cost
Leaf-level defragmentation	Rebuilds tree
Online	Online depends on supported options
Does not refresh index stats	Refreshes index stats
Useful lower-overhead option	Use when rebuild is justified

Rebuild All Indexes Example

```sql
ALTER INDEX ALL
ON fundraising.Gift
REBUILD;
```

Do **not** run this routinely just because it exists.

On a large production table this can be expensive.

Reorganize All Indexes Example

```sql
ALTER INDEX ALL
ON fundraising.Gift
REORGANIZE;
```

Again:

> ! Measure first.

Index Maintenance Is Not the Same as Statistics Maintenance

This distinction matters.

Suppose performance improves after:

```sql
ALTER INDEX ... REBUILD
```

You might assume:

```
Fragmentation was the problem.
```

But rebuild also refreshed index statistics.

The real benefit might have come from:

```
better cardinality estimates
```

rather than reduced fragmentation.

Microsoft specifically warns that some apparent rebuild benefits are actually due to updated statistics, and updating statistics can sometimes achieve the benefit at much lower cost.

Update Statistics Directly

Example:

```sql
UPDATE STATISTICS
    fundraising.Gift;
```

or database-wide:

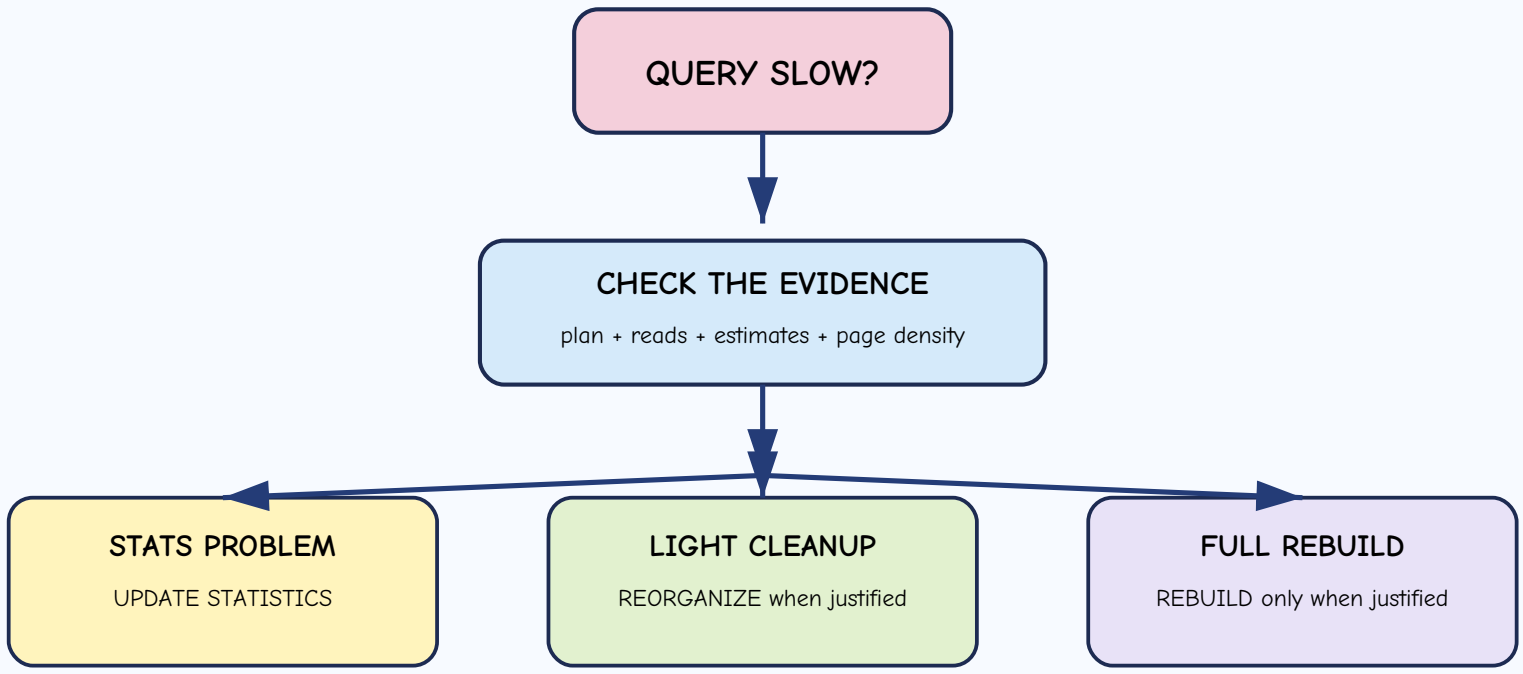
```sql
EXEC sys.sp_updatestats;
```

SQL Server already supports automatic statistics updates, so manual updates should be used thoughtfully rather than constantly.

Index Maintenance Decision Process

FLOW DIAGRAM

Index Maintenance - Measure Before You Rebuild



Fragmentation percentage alone is not a maintenance command. Consider page count, density, statistics, query behavior, and workload impact first.

Better thinking:

EXAMPLE / FLOW

```
Query slow?
    |
    v
Inspect actual plan
    |
    v
Check logical reads
    |
    v
Check estimates/statistics
    |
    v
Check index usage
    |
    v
Check page density/fragmentation
    |
    v
Determine actual cause
```

Not:

EXAMPLE / FLOW

```
Query slow
    |
    v
REBUILD ALL INDEXES
```

Index Creation Naming

A useful naming convention might be:

EXAMPLE / FLOW

```
CX_
→ clustered index

IX_
→ normal nonclustered index

UX_
→ unique index
```

Examples:

EXAMPLE / FLOW

```
CX_Gift_GiftId

IX_Gift_Tenant_Constituent

UX_Campaign_Tenant_CampaignCode
```

Naming conventions vary by team.

Consistency matters more than the exact prefix.

Creating a Normal Nonclustered Index

SQL

```
CREATE INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
);
```

Because `NONCLUSTERED` is the default for a normal `CREATE INDEX`, this could also be explicitly written:

SQL

```
CREATE NONCLUSTERED INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift
(
    ConstituentId
);
```

Dropping an Index

SQL

```
DROP INDEX
    IX_Gift_ConstituentId
ON fundraising.Gift;
```

Be careful before dropping indexes.

Check:

EXAMPLE / FLOW

```
Workload

Dependencies

Usage

Query Store

Execution plans
```

Viewing Table Indexes

SQL

```
SELECT
    i.index_id,
    i.name,
    i.type_desc,
    i.is_unique,
    i.is_primary_key,
    i.is_unique_constraint,
    i.has_filter,
    i.filter_definition,
    i.fill_factor

FROM sys.indexes AS i

WHERE i.object_id =
    OBJECT_ID
    (
        'fundraising.Gift'
    )

ORDER BY
    i.index_id;
```

`sys.indexes` exposes properties such as index type, uniqueness, filtering, and fill factor.

Viewing Index Columns

SQL

```
SELECT
    i.name AS IndexName,
```

```
    c.name AS ColumnName,

    ic.key_ordinal,

    ic.is_included_column

FROM sys.indexes AS i

INNER JOIN sys.index_columns AS ic
    ON ic.object_id = i.object_id
   AND ic.index_id  = i.index_id

INNER JOIN sys.columns AS c
    ON c.object_id = ic.object_id
   AND c.column_id = ic.column_id

WHERE i.object_id =
      OBJECT_ID
      (
          'fundraising.Gift'
      )

ORDER BY
    i.name,
    ic.key_ordinal,
    ic.index_column_id;
```

This helps identify:

EXAMPLE / FLOW

```
key columns

included columns

column order
```

Practical Index Design - Get Constituent Gifts

Query:

SQL

```
SELECT
    GiftId,
    GiftDate,
    TotalAmount
FROM fundraising.Gift
WHERE TenantId = @TenantId
  AND ConstituentId = @ConstituentId
ORDER BY
    GiftDate DESC,
    GiftId DESC;
```

Possible learning index:

SQL

```
CREATE INDEX
    IX_Gift_Tenant_Constituent_GiftDate
ON fundraising.Gift
(
    TenantId,
```

```
    ConstituentId,
    GiftDate DESC,
    GiftId DESC
)
INCLUDE
(
    TotalAmount
);
```

Why This Index?

Search:

EXAMPLE / FLOW

```
TenantId
ConstituentId
```

Sort:

EXAMPLE / FLOW

```
GiftDate DESC
GiftId DESC
```

Output:

EXAMPLE / FLOW

```
TotalAmount
```

Therefore:

EXAMPLE / FLOW

```
TenantId
ConstituentId
GiftDate
GiftId
```

are key columns.

`TotalAmount` can be included.

Does This Mean You Must Create It?

No.

This is a learning candidate.

In a real system first check:

EXAMPLE / FLOW

```
existing indexes

actual execution plan

query frequency
```

```
write workload

logical reads
```

Practical Index Design - Open Tasks

Query:

SQL

```
SELECT
    TaskId,
    ConstituentId,
    DueDate,
    Priority
FROM engagement.Task
WHERE TenantId = @TenantId
  AND AssignedToUserId = @UserId
  AND CompletedAtUtc IS NULL
ORDER BY DueDate;
```

Possible learning filtered index:

SQL

```
CREATE INDEX
    IX_Task_Open_AssignedUser
ON engagement.Task
(
    TenantId,
    AssignedToUserId,
    DueDate
)
INCLUDE
(
    TaskId,
    ConstituentId,
    Priority
)
WHERE CompletedAtUtc IS NULL;
```

This combines:

EXAMPLE / FLOW

```
Composite

Filtered

Covering

Included columns
```

Practical Index Design - Campaign Code Uniqueness

Requirement:

! CampaignCode should be unique within each tenant.

```sql
CREATE UNIQUE INDEX
    UX_Campaign_Tenant_CampaignCode
ON fundraising.Campaign
(
    TenantId,
    CampaignCode
);
```

This enforces:

```
Tenant 1 + CANCER2026
→ one Campaign only
```

while:

```
Tenant 2 + CANCER2026
```

could still exist.

Practical Index Design - Gift Allocations

Query:

```sql
SELECT
    FundId,
    AllocationAmount
FROM fundraising.GiftAllocation
WHERE GiftId = @GiftId;
```

Candidate:

```sql
CREATE INDEX
    IX_GiftAllocation_GiftId
ON fundraising.GiftAllocation
(
    GiftId
)
INCLUDE
(
    FundId,
    AllocationAmount
);
```

This can potentially cover:

```
find allocations for Gift
```

without needing extra base-row retrieval.

Practical Index Design - Fund Donation Reporting

Query:

```sql
SELECT
    ga.GiftId,
    ga.AllocationAmount
FROM fundraising.GiftAllocation AS ga
WHERE ga.FundId = @FundId;
```

A separate index beginning with:

```
FundId
```

may be needed.

Why?

Because an index:

```
(GiftId, FundId)
```

is optimized first by:

```
GiftId
```

and may not efficiently seek by `FundId` alone.

This demonstrates why:

```
column order matters.
```

More Index Types You Should Know Exist

Your current chapter focuses mainly on traditional rowstore indexes.

SQL Server also supports specialized indexing technologies such as:

```
Columnstore indexes

XML indexes

Spatial indexes

Full-text indexes

Indexes on computed columns

Indexes on views

Memory-optimized table indexes
```

Do not mix these into the current rowstore concepts.

For your learning order:

EXAMPLE / FLOW

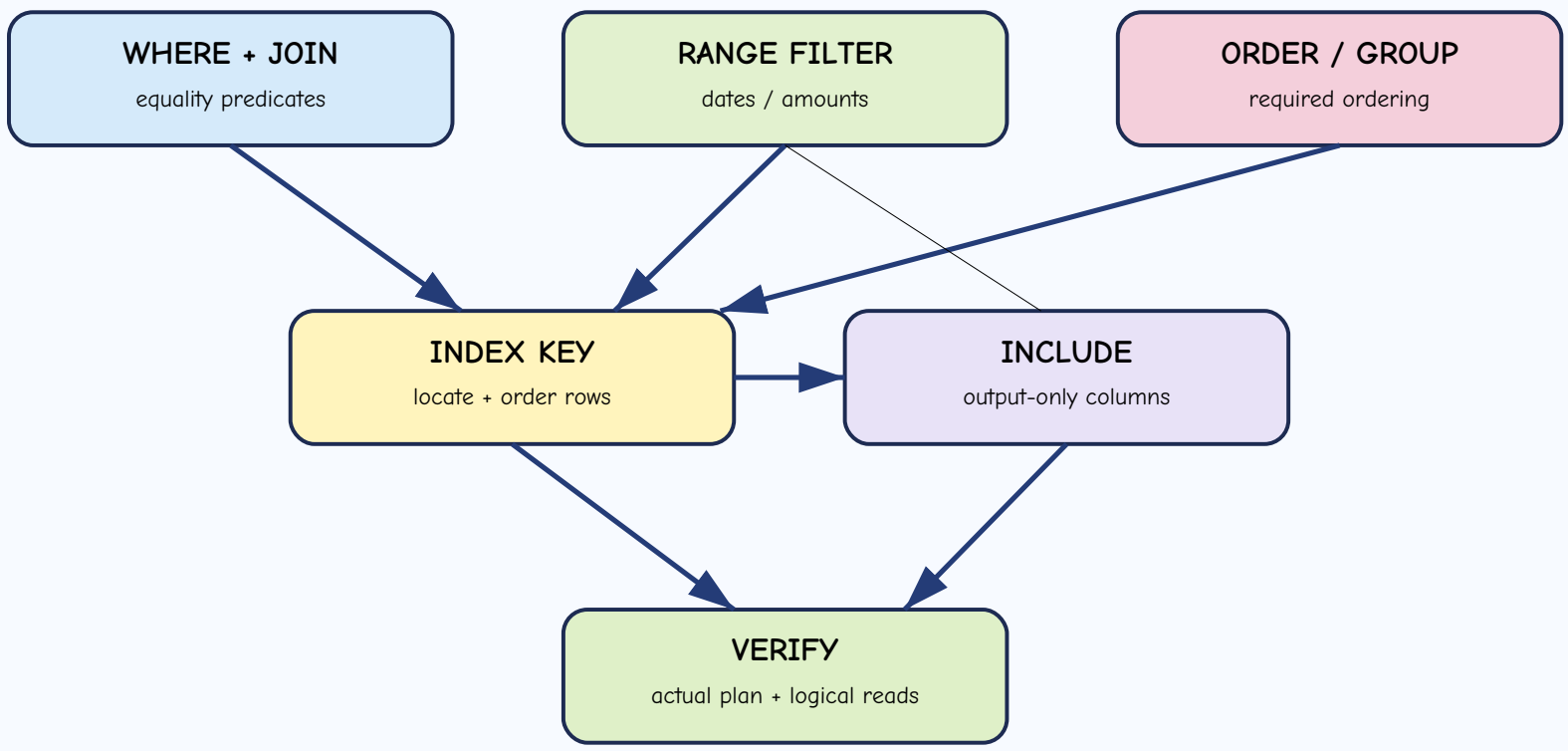
```
Clustered / Nonclustered
        ↓
Composite / Unique
        ↓
Filtered / Covering / INCLUDE
        ↓
Seek / Scan / Lookup
        ↓
Maintenance
        ↓
Then specialized index types
```

is much easier.

Index Selection Rules for Beginners

FLOW DIAGRAM

Start from the Query - Build the Index Key Intentionally



Equality columns often lead the key, range/order columns follow when useful, and output-only values are good INCLUDE candidates. Then verify the design with evidence.

When considering an index, ask:

EXAMPLE / FLOW

```
1. Which columns appear in WHERE?

2. Which columns participate in JOIN?

3. Which columns appear in ORDER BY?

4. Which columns appear in GROUP BY?
```

```
5. Which columns only need to be returned?

6. How selective are the search columns?

7. How frequently is the query executed?

8. How frequently is the table modified?

9. Does a similar index already exist?

10. Can one well-designed index serve several queries?
```

Equality Search Columns

Example:

```sql
WHERE TenantId = 1
  AND ConstituentId = 1001
```

These are strong index-key candidates when the workload frequently searches this way.

Range Search Columns

Example:

```sql
GiftDate >= @StartDate
AND GiftDate < @EndDate
```

`GiftDate` can be useful as a key after the main equality columns.

Output-Only Columns

Example:

```sql
SELECT
    TotalAmount,
    CampaignId
```

but not used in:

```
WHERE
JOIN
ORDER BY
GROUP BY
```

These may be candidates for:

```
INCLUDE
```

rather than key columns.

Low-Selectivity Columns

Suppose:

EXAMPLE / FLOW
```
Status
```

contains only:

EXAMPLE / FLOW
```
Active
Inactive
```

and almost all rows are:

EXAMPLE / FLOW
```
Active.
```

A full index on:

EXAMPLE / FLOW
```
Status
```

alone may provide little value.

However a filtered index on a small important subset can sometimes be excellent.

Example:

EXAMPLE / FLOW
```
CompletedAtUtc IS NULL
```

where only 2% of rows are open.

Indexes and Foreign Keys

SQL Server does not automatically create an index on the child foreign-key column merely because you create a foreign key.

Example:

EXAMPLE / FLOW
```
Gift.ConstituentId
→ Constituent.ConstituentId
```

The foreign key protects integrity.

An index on:

EXAMPLE / FLOW
```
Gift.ConstituentId
```

is a separate performance decision.

Foreign-key columns are often good candidates for indexes when parent/child queries or deletes/updates frequently use them.

Common Index Mistakes

Index Every Column

More indexes mean more write and storage overhead.

Do not create:

EXAMPLE / FLOW

```
IX_Gift_GiftType

IX_Gift_Currency

IX_Gift_Status

IX_Gift_Notes

IX_Gift_Everything
```

without workload evidence.

Assume Primary Key Must Be Clustered

It often is by default.

But:

EXAMPLE / FLOW

```
PK
```

and:

EXAMPLE / FLOW

```
clustered index
```

are separate design concepts.

Assume Clustered Index Guarantees Output Order

It does not.

Use:

SQL

```
ORDER BY
```

Wrong Composite Index Order

Index:

EXAMPLE / FLOW

```
(GiftDate, TenantId, ConstituentId)
```

may not efficiently serve the same searches as:

EXAMPLE / FLOW

```
(TenantId, ConstituentId, GiftDate)
```

Key order matters.

Put Every Output Column in the Key

Use:

EXAMPLE / FLOW

```
INCLUDE
```

for output-only columns when appropriate.

Make Covering Indexes Extremely Wide

Covering is useful.

Over-covering can become expensive.

Remove Every Key Lookup

A lookup for:

EXAMPLE / FLOW

```
3 rows
```

may be cheaper than maintaining a much larger index permanently.

Treat Index Scan as Failure

A scan may be completely correct.

Treat Index Seek as Automatically Excellent

A seek retrieving millions of rows can still be expensive.

Ignore Filtered Index Opportunities

Small important subsets such as:

EXAMPLE / FLOW

```
Open tasks

Unprocessed messages

Non-NULL optional business identifiers
```

can sometimes benefit greatly.

Use Global Low Fill Factor Everywhere

Lower fill factor wastes page space when it is not needed.

Use it only when measured page splits justify it.

Rebuild Every Index Every Night

This can create huge unnecessary:

EXAMPLE / FLOW

```
CPU

I/O

Transaction log

Storage

Blocking/resource pressure
```

Modern guidance favors targeted, measured maintenance.

Focus Only on Fragmentation

Also examine:

EXAMPLE / FLOW

```
Page density

Statistics

Logical reads

Actual execution plans

Query Store

Workload impact
```

Rebuild When Updating Statistics Would Solve the Problem

Rebuild refreshes index statistics too.

Sometimes the improvement attributed to rebuild actually came from statistics refresh.

Ignore Write Cost

Every extra index must be maintained when data changes.

Always balance:

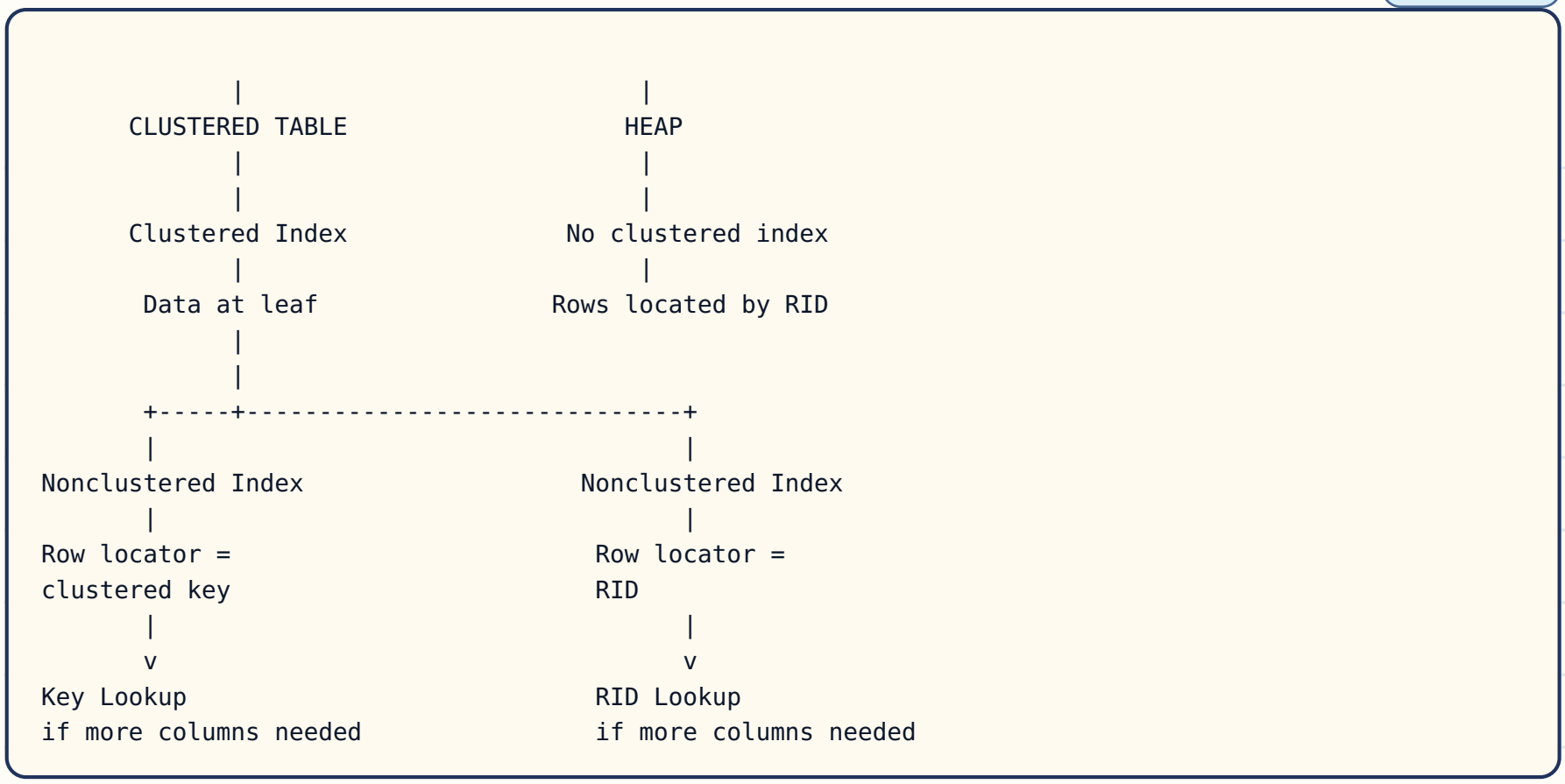
EXAMPLE / FLOW

```
Read performance
vs
Write cost
```

Master Diagram

EXAMPLE / FLOW





Master Index Type Comparison

Concept	Meaning
Clustered index	Data rows stored at leaf level by clustered key
Nonclustered index	Separate searchable structure
Composite index	Multiple key columns
Unique index	Prevents duplicate key values
Filtered index	Indexes only selected rows
Covering index	Contains everything one query needs
INCLUDE	Adds nonkey leaf columns
Heap	Table without clustered index

Execution Operator Memory Table

Operator	Meaning
Index Seek	Navigate to matching index key/range
Index Scan	Read broad/all index
Clustered Index Seek	Seek directly in clustered table
Clustered Index Scan	Scan clustered table
Key Lookup	Fetch missing columns from clustered row
RID Lookup	Fetch missing row from heap
Table Scan	Scan a heap

Fill Factor Memory

EXAMPLE / FLOW

```
FILLFACTOR 100/0
→ pages built very full

Lower fill factor
→ reserve more free space

More free space
→ potentially fewer middle page splits

But

More free space
→ larger index
→ more memory/I/O
```

Maintenance Memory

EXAMPLE / FLOW

```
REORGANIZE
→ lighter incremental leaf cleanup
```

EXAMPLE / FLOW

```
REBUILD
→ recreate index
```

EXAMPLE / FLOW

```
UPDATE STATISTICS
→ refresh optimizer distribution information
```

These solve related but not identical problems.

Interview Questions

What is an index?

An index is a data structure that helps SQL Server locate rows more efficiently.

It can reduce the amount of data SQL Server must examine, but it also adds storage and write-maintenance overhead.

What is a clustered index?

A clustered index stores the table's data rows at the leaf level according to the clustered key.

A table can have only one clustered index.

Why can a table have only one clustered index?

Because the table's rows themselves form the leaf level of that clustered structure and can have only one clustered storage arrangement.

What is a nonclustered index?

A nonclustered index is a separate structure containing key values and a row locator pointing to the base row.

What is the row locator in a clustered table?

The clustered index key.

What is the row locator in a heap?

A RID containing the file, page, and slot information.

What is a composite index?

An index with more than one key column.

Example:

SQL

```
CREATE INDEX IX_Gift_Tenant_Constituent
ON fundraising.Gift
(
    TenantId,
    ConstituentId
);
```

Why does column order matter in a composite index?

Because SQL Server's index structure is ordered from the first key onward.

An index on:

EXAMPLE / FLOW

```
(TenantId, ConstituentId)
```

is naturally searchable by `TenantId`, but it is not equivalent to an index beginning with `ConstituentId`.

What is a unique index?

A unique index prevents duplicate values for the index-key combination and also gives the optimizer information about uniqueness.

Can a unique index contain NULL?

Yes, but SQL Server treats NULL as a value for uniqueness enforcement.

For a single-column unique index, normally only one NULL is permitted.

What is a filtered index?

A filtered index is a nonclustered index containing only rows satisfying a filter predicate.

Example:

```sql
CREATE INDEX IX_Task_Open
ON engagement.Task
(
    TenantId,
    AssignedToUserId
)
WHERE CompletedAtUtc IS NULL;
```

What is a covering index?

A covering index contains all columns required by a particular query, allowing SQL Server to satisfy the query without accessing the base table/clustered index for additional columns.

What is INCLUDE?

`INCLUDE` adds nonkey columns to the leaf level of a nonclustered index.

Use it mainly for columns needed by query output but not needed for searching/order.

Key column vs included column?

Key column:

```
participates in index ordering/search structure.
```

Included column:

```
stored only at leaf level to help cover queries.
```

What is an Index Seek?

An Index Seek navigates through an index to retrieve specific matching key values or ranges.

What is an Index Scan?

An Index Scan reads a broad range or all rows of an index.

It can be the correct operator when many rows are required.

Is an Index Seek always better than a Scan?

No.

A seek returning millions of rows may cost more than an efficient scan.

Always evaluate the complete plan and row counts.

What is a Key Lookup?

A Key Lookup retrieves missing columns from the clustered index after a nonclustered index has located rows.

What is a RID Lookup?

A RID Lookup retrieves the base row from a heap using its row identifier.

What is a heap?

A heap is a table without a clustered index.

It stores rows without clustered-key ordering and uses RIDs to identify rows.

What is fill factor?

Fill factor controls how full index leaf pages are made when the index is created or rebuilt.

Lower values reserve additional free space for future changes but also increase storage and I/O requirements.

What is index fragmentation?

Fragmentation describes inefficient page organization.

Important measurements include:

EXAMPLE / FLOW

```
logical fragmentation

page density
```

Modern SQL Server tuning should consider both rather than fragmentation percentage alone.

Difference between REORGANIZE and REBUILD?

`REORGANIZE` is a lighter incremental leaf-level defragmentation operation.

`REBUILD` recreates the index and is more resource intensive.

A rebuild refreshes index statistics; reorganize does not.

Should every fragmented index be rebuilt?

No.

Consider:

EXAMPLE / FLOW

```
page count

page density

query performance

logical reads

statistics

workload impact
```

before deciding.

Microsoft specifically recommends not assuming index maintenance will automatically improve workload performance.

Full Interview Problem

Requirement

This query is common:

SQL

```sql
SELECT
    g.GiftId,
    g.GiftDate,
    g.TotalAmount

FROM fundraising.Gift AS g

WHERE g.TenantId = @TenantId
  AND g.ConstituentId = @ConstituentId
  AND g.GiftDate >= @StartDate
  AND g.GiftDate < @EndDate

ORDER BY
    g.GiftDate DESC,
    g.GiftId DESC;
```

How would you think about the index?

Step 1 - Equality Filters

EXAMPLE / FLOW

```
TenantId

ConstituentId
```

These are natural leading key candidates.

Step 2 - Range

EXAMPLE / FLOW

```
GiftDate
```

comes after the equality predicates.

Step 3 - Ordering

The application wants:

EXAMPLE / FLOW

```
GiftDate DESC
GiftId DESC
```

so those columns may participate in the key.

Step 4 - Output

EXAMPLE / FLOW

```
TotalAmount
```

is returned but isn't required to locate the rows.

It may be an included column.

Candidate Learning Index

SQL

```
CREATE INDEX
    IX_Gift_Tenant_Constituent_Date
ON fundraising.Gift
(
    TenantId,
    ConstituentId,
    GiftDate DESC,
    GiftId DESC
)
INCLUDE
(
    TotalAmount
);
```

Why?

EXAMPLE / FLOW

```
TenantId
→ equality search

ConstituentId
→ equality search

GiftDate
→ range + ordering

GiftId
→ deterministic ordering

TotalAmount
→ output-only included value
```

This index could potentially provide:

EXAMPLE / FLOW

```
Index Seek
+
ordered rows
+
```

```
covered query
+
no Key Lookup
```

But before creating it in a real system, inspect:

EXAMPLE / FLOW

```
existing indexes

actual execution plan

logical reads

query frequency

write overhead
```

That final step is essential.

Final Revision Map

EXAMPLE / FLOW

```
SQL SERVER INDEXES
        |
        +---- Clustered
        |       |
        |       +---- Data at leaf
        |       +---- One per table
        |
       +---- Nonclustered
        |       |
        |       +---- Separate structure
        |       +---- Row locator
        |
       +---- Design Variations
        |       |
        |       +---- Composite
        |       +---- Unique
        |       +---- Filtered
        |       +---- Covering
        |       +---- INCLUDE
        |
       +---- No Clustered Index
        |       |
        |       +---- Heap
        |       +---- RID
        |
       +---- Execution
        |       |
        |       +---- Seek
        |       +---- Scan
        |       +---- Key Lookup
        |       +---- RID Lookup
        |
```

```
+---- Page Management
|       |
|       +---- Fill Factor
|       +---- Page Split
|       +---- Page Density
|       +---- Fragmentation
|
+---- Maintenance
        |
        +---- REORGANIZE
        +---- REBUILD
        +---- UPDATE STATISTICS
```

Final Easy Memory Rules

EXAMPLE / FLOW
Index
→ Faster path to data

EXAMPLE / FLOW
Clustered Index
→ Data is the leaf

EXAMPLE / FLOW
Nonclustered Index
→ Separate structure pointing to data

EXAMPLE / FLOW
Composite Index
→ Multiple key columns

EXAMPLE / FLOW
Unique Index
→ No duplicate key combination

EXAMPLE / FLOW
Filtered Index
→ Only selected rows are indexed

EXAMPLE / FLOW
Covering Index
→ Everything one query needs is in the index

EXAMPLE / FLOW
INCLUDE
→ Output/support columns at leaf level

EXAMPLE / FLOW
Heap
→ Table without clustered index

EXAMPLE / FLOW
Index Seek
→ Navigate to target rows

EXAMPLE / FLOW
Index Scan
→ Read broad/all index

EXAMPLE / FLOW
Key Lookup
→ Go from nonclustered index to clustered row

EXAMPLE / FLOW
RID Lookup
→ Go from nonclustered index to heap row

EXAMPLE / FLOW
Fill Factor
→ Free space reserved during index build/rebuild

EXAMPLE / FLOW
Page Split
→ Full page must be split to make space

EXAMPLE / FLOW
Fragmentation
→ Pages become less efficiently organized

EXAMPLE / FLOW
Page Density
→ How full pages are

EXAMPLE / FLOW
REORGANIZE
→ Lower-cost incremental index cleanup

EXAMPLE / FLOW

```
REBUILD
→ Re-create index
```

EXAMPLE / FLOW

```
UPDATE STATISTICS
→ Refresh optimizer knowledge about data
```

The most important index-design rule is:

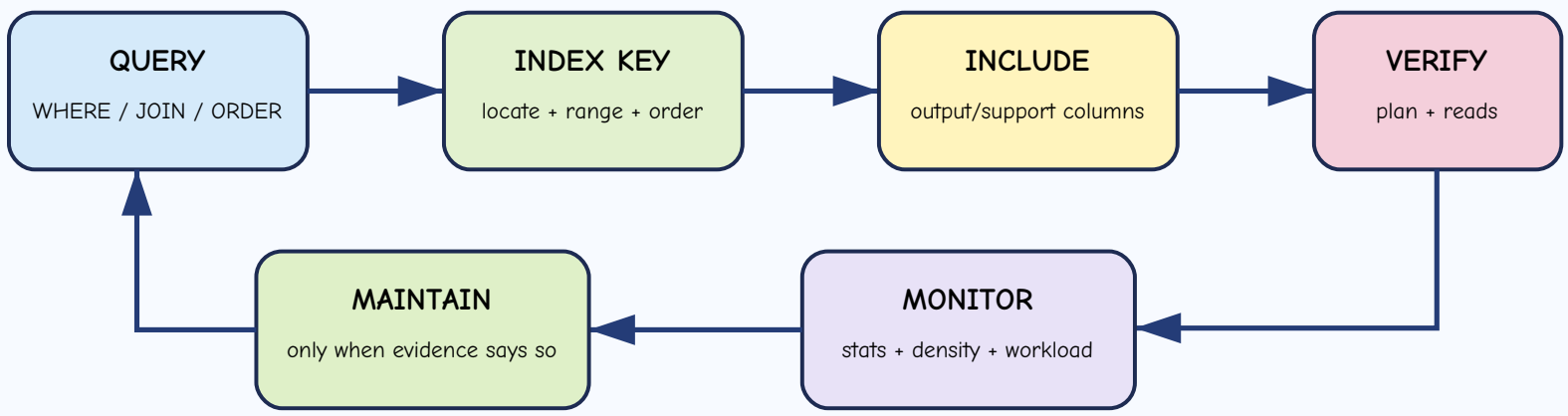
> ! **Start from the query, not from the table. Put columns used to locate, join, range-filter, or order rows into the index key in a useful order; use included columns for values needed mainly as output; then verify the design using the actual execution plan, logical reads, and the real workload.**

And the most important maintenance rule is:

> ! **Do not rebuild indexes simply because a fragmentation percentage crosses an old fixed threshold. Measure query performance, page density, page count, statistics quality, and workload impact first, then choose no action, statistics maintenance, reorganize, or rebuild based on evidence.**

FINAL FLOW

Index Design and Maintenance Evidence Loop



Start from the workload, validate the access path, then maintain only the structures that matter.

The same evidence-based loop applies to both index creation and index maintenance.